

here you can see an example LORA frame and in the beginning you see the standard stuff like preamble which is just all zeros and so this one little sawtooth wave is just one symbol and this repeats for a number of times and then you have the frame sync and frequency sink was functions just to make sure that the receiver is synchronized in time and frequency to the transmitter and then we have a header which basically encodes some information about the coding rate used whether there is a CRC in the payloads and then we have the payloads itself and actually in the case of LORA there can be some symbols from the payload in the header as well so this frame structure can be easily derived from the patent and here's the link if you're interested it also contains information on how exactly they encode data into this format so the modulation and interleaving some other information is located in data sheets for example whitening and coding is not covered in the patents but it's somewhat discussed in the white papers so with that let's start building a receiver now usually the first step that you would do is detect the signal and detecting is a pretty standard problem in signal processing so you can just really any algorithm that works for you for me I chose this algorithm which exploits the autocorrelation of repeating preamble so you just take the auto correlation between these two for example and then you will see that as the symbols get more similar then you can see the metric rising and then it approaches one for example but now we still need to find out what the start is of a single symbol so I drew these red lines on the graph and those are drawn by myself so we still have to find where this position is exactly and if you would thresholds on this metric then that would probably be a very bad approach because you would end up something around here or around here which is not ideal so trash holding is bad and to synchronize again you have multiple possibilities I saw some people that do modulate the preamble symbol which is supposed to be 0 and therefore if you have an offset from zero then that indicates a time shift which is the basic principle of Laurel mutilation but in that case you would have some ambiguity because there is if you have a frequency shift and that will also cause offset from zero so what I chose to do instead is I just cross correlates with the instantaneous frequency and then you get a metric that looks like this so you locally generates a preamble yourself and then cross correlated and it will be near or close to 1 at the start of the symbol so with that you can synchronize to single symbol okay now the next question is how do we demodulates a single symbol so the modulation of LORA is based on CSS and it uses chirps so chirp is basically just signal that's linearly increases in frequency over time so it starts at the negative frequency then you can see it increasing until it reaches zero and then you can see the zero here and then its increases back so to modulate a value I for example on to the chirp what you would do is he would psychically shift it so in this example you can see the set the signal shifted to the left and this will result in something that looks like this and for this example the value is 20 but normally you wouldn't know that because the patent is very secretive about how with this shift is really mapped to real value but now we have seen how to modulate it but how do we give modulate so you when you receive a lower signal he receive chirp so that already shifted so we need to do the reverse operation and that can be done by multiplying by the conjugate base chirp so just the base ship with the imaginary parts negated and then we sample it the chirp rate and the details are not important but it boils down to when you do that and you take the FFT then you get a peak in the frequency domain where the modulated value should be so for example for the preamble which is always zero three M symbols are always zero so the peak you will get is at index zero for the shift its symbol there's something strange going on because you can see that

it's not exactly 2 and it's 24 and that's because this in axis what's called gray codes okay so after the demodulation you have basically a list of chip values and now we need to do some other things to get to our ultimate data the interleaving algorithm is described in the LORA patent as well and it's dependent on two configurations the first is a spreading factor which determines the number of bits per symbol values so here you can see the spreading factor in this direction in this case the spreading factor is seven and then there is the coding rate which determines how many symbols there are in the interleaving matrix so this should correspond to the coding rate so in this case it's eight since we use four eighth coding so now if you look at this matrix what LORA does it essentially transposes this matrix and then walks through it in a diagonal fashion so for example for the last code words you would just start here then move upwards in a diagonal way and when you reach the top you just continue from the bottom and this is pretty easy to implement in software so you can just do this for the entire matrix and you will end up with a list of the interleaved code words now as you may notice the advantage of using this technique is that if you have for example this symbol right here if it gets entirely corrupted then it turns out that only the 30 last column of bits will be corrupted and this can be corrected for since we use timing coding so we can just correct this code words now what's left to be done so in the previous slides I told you guys that there that the index is great code it means how would you reverse engineer yet because it's not in the patent or yeah it mentions grain coding but it's not clear whether this is which index corresponds to which integer value and also at which stage of the decoding is weighting performed and how that's also something that's not known there are some more to it for example the structure of the header that's used lower around the clock drift correction and then there are some weird stuff like LORA appears to swap nibbles which is something that GSM also does if I'm correct and it's some weird CRC so we don't we won't have a time to discuss that in this presentation so anyway on to the relation between the symbol and the integer value so here we have our example again with $L = 20$ with a certain shift and there are multiple ways to interpret this for example you could interpret the x-axis as this so this is equal to 24 but then the question is do we use gray encoding or gray decoding to get the real value of L and similarly we can interpret the x-axis as inverted so in that case this would be 103 or we can do degree of 103 so check for correctness what we can do is you can implement the decoder up to the interleaving stage and start looking for patterns if there is some pattern that doesn't match up with other payloads then you know that there is something wrong with your assumptions so let's take a look at an example so in the top left quadrants I have printed out all headers symbols when using right-to-left indexing so 127 to 0 and using gray encoding to get the value of the symbol so I'm gonna give you some time to look at this and let's see if you can figure out where the length is used here is the length at L so this is something I added myself this is just the length of the packet that I sent so you should see some pattern that stays the same here and then increases here and again stays the same here and then increases in the lower nibble and so if you look closely you can see that this pattern appears to emerge here because you have this looks like 0 and then it's 1 2 3 then you have here 1 and again 0 1 2 and the ones the Tuesday even correspond but there's something strange going on because I said before that the header is not white and it appears to be that's in this case for example a 2 is not equal to 2 it's just all zeros so there's something really weird going on and also if you take a 2 from the highest level and go to the lower level then you can see that I don't match so that's not correct then let's go to gray decoding in that case it's even weirder because in this case you'll have

some bits that are added or removed so this indicates that there is something really wrong with the previous layers of your decoder so we can actually discard this whole column right here and the right solution turns out to be left-to-right indexing and gray encoding and in this case you would get like you would expect expect that the 0 is effectively 0 as a Hamming code and it's consistent with the highest and lowest nibble the only weird thing is that the order chain so first we had two lengths here and now it appears in these two columns all right so now that we have reversed the interleaving and all the other aspects there are still two more aspects to reverse and that's the coding but since we use four five coding and four eight coding as options that's already a strong indication that probably Hamming coding is used but those of you who know about Hamming coding might notice something strange because the indexing doesn't really or the end the size of the bits don't really match up with standards Hamming coding and also it turns out the payload is white and so that means X sort with a random string basically and this is to make the data more uniform they mention the algorithm used for that in a datasheet but this doesn't appear to work in practice so in that case you need to answer another question and that's in what stage is this data widened all right so the first solution for me was to just brute-force out possibilities for the whitening sequence what you can do is you can just send a payload with all zeros because if you XOR something with all zeros and you just get the value itself so in that case when the result of our payload is all zeros we just end up with getting the whitening sequence which we can store in an array and then use for future packets now if anyone has an idea of how to do this more algebraically I would be happy to learn it because there is a library somewhere that seems to have an algorithm for generating it algebraically but the writing sequence doesn't really match up with my own and also if you try to decode you get some weird values sometimes so and then there is Hamming code so as I said before the finishes are not equal to standards Hamming so they are permuted and so we can just go over all possible permutations of a single byte which is not that hard to calculate so it's only this amount of possibilities and it was pretty easy to brute-force alright so now we have all of the required components linked together in order to build a fully functional decoder so we have just are we amulet action sync modulation and leaving lightning coding and then we have our raw data I have implemented the decoder for this and it's also open source on github if you want to check it out feel free to do so I made a comparison with real hardware for various number of payloads and as you can see there is it's still quite a lot worse so there's still some work to be done for example the real lower hardware is capable of even decoding packets that go below the noise floor so where is my because of and that's because of the methods are at choice for the preamble sink and or PML detection and synchronization so there's still some work to be done here an advantage of my approach is that it's virtually not affected by frequency errors so and that's yeah because of the gradient decoding that I implemented but essentially this allows you to decode multiple channels at the same time and that was useful for my research alright so special thanks also to my students William for implementing some optimizations there are some other decoders out there which you should also check out there's LORA SDR and then a busy research is Gia LORA and so maybe they will work better for your use case all the pens I guess and then let's move on to doing something with the decoder so let's go to our first application which will be fingerprinting LORA devices using neural networks now you might wonder why would you want to fingerprint devices and people usually give us the advantages in the defensive use cases for example an extra layer of defense for critical infrastructure suppose you